

STATS 506 Homework2

Wei min

Problem 1: Random Walk with Momentum

(a)

I keep track of the current direction and add each step to the position. When a new direction is drawn, it can be the same as the previous one.

```
momentum_walk <- function(n, p) {
  stopifnot(length(n) == 1, is.numeric(n), is.finite(n),
            n >= 1, n == floor(n),
            length(p) == 1, is.numeric(p), is.finite(p),
            p >= 0, p <= 1)

  steps <- rbind(c(1, 0), c(-1, 0), c(0, 1), c(0, -1))
  direction <- sample(1:4, 1)
  position <- c(0, 0)

  for (i in seq_len(n)) {
    if (i > 1 && runif(1) >= p) {
      direction <- sample(1:4, 1)
    }
    position <- position + steps[direction, ]
  }
  position
}

set.seed(1101)
momentum_walk(10, 0.6)
```

```
[1] 2 -4
```

`stopifnot()` checks that `n` is a positive integer and `p` is between 0 and 1. The output is the final position, (`x`, `y`).

(b)

The function above uses a loop, so I use it as `momentum_walk1`.

```
momentum_walk1 <- momentum_walk

momentum_walk2 <- function(n, p) {
  stopifnot(length(n) == 1, is.numeric(n), is.finite(n),
            n >= 1, n == floor(n),
            length(p) == 1, is.numeric(p), is.finite(p),
            p >= 0, p <= 1)

  steps <- rbind(c(1, 0), c(-1, 0), c(0, 1), c(0, -1))
  direction <- sample(1:4, 1)
```

```

moves <- sapply(seq_len(n), function(i) {
  if (i > 1 && runif(1) >= p) {
    direction <- sample(1:4, 1)
  }
  steps[direction, ]
})

rowSums(moves)
}

set.seed(1102)
momentum_walk1(10, 0.6)

```

```
[1] 1 -1
```

```
momentum_walk2(10, 0.6)
```

```
[1] 1 -1
```

```
momentum_walk1(1000, 0.6)
```

```
[1] 51 -45
```

```
momentum_walk2(1000, 0.6)
```

```
[1] -30 46
```

The loop adds each step directly to the position. The second version uses `sapply()` to store the steps, then adds them with `rowSums()`. Here, `<-` updates `direction` inside `momentum_walk2` so it carries over to the next step.

(c)

```

set.seed(1103)
momentum_walk1(10, 0.6)

```

```
[1] -10 0
```

```

set.seed(1103)
momentum_walk1(10, 0.6)

```

```
[1] -10 0
```

```

set.seed(1104)
momentum_walk1(1000, 0.6)

```

```
[1] 136 44
```

```

set.seed(1104)
momentum_walk1(1000, 0.6)

```

```
[1] 136 44
```

Each pair gives the same result. Resetting the seed makes R use the same sequence of random numbers again.

(d)

```

set.seed(1105)
p_values <- c(0, 0.5, 0.9)

distances <- sapply(p_values, function(p) {
  position <- momentum_walk1(1000, p)

```

```

    sqrt(sum(position^2))
  })

data.frame(p = p_values, distance = distances)

```

```

  p distance
1 0.0  50.53712
2 0.5  56.30275
3 0.9 243.82371

```

The distances are about 50.54, 56.30, and 243.82. They increase with momentum in these runs, but one walk at each value of p is not enough to tell whether this usually happens.

(e)

```

set.seed(1106)
p_values <- c(0, 0.25, 0.5, 0.75, 0.9)

mean_distance <- sapply(p_values, function(p) {
  distances <- replicate(1000, {
    position <- momentum_walk1(1000, p)
    sqrt(sum(position^2))
  })
  mean(distances)
})

data.frame(p = p_values, mean_distance = mean_distance)

```

```

  p mean_distance
1 0.00      27.90538
2 0.25      36.20723
3 0.50      47.82017
4 0.75      76.43849
5 0.90     124.32563

```

The mean distance increases from about 27.9 at $p = 0$ to 124.3 at $p = 0.9$. With higher momentum, the walk stays in the same direction longer, so it usually ends farther from the origin.

(f)

```

set.seed(1107)
small <- microbenchmark(
  loop = momentum_walk1(1000, 0.6),
  apply = momentum_walk2(1000, 0.6),
  times = 30
)
large <- microbenchmark(
  loop = momentum_walk1(100000, 0.6),
  apply = momentum_walk2(100000, 0.6),
  times = 30
)

timing <- rbind(
  data.frame(n = 1000, summary(small, unit = "ms")[, c("expr", "median")]),
  data.frame(n = 100000, summary(large, unit = "ms")[, c("expr", "median")])
)

```

```
names(timing) <- c("n", "method", "median_ms")
knitr::kable(timing, digits = 3, row.names = FALSE)
```

n	method	median_ms
1e+03	loop	0.966
1e+03	apply	1.234
1e+05	loop	103.651
1e+05	apply	131.483

The loop is faster at both sizes in this run, and the time gap is larger at 100,000 steps. `sapply()` calls a function for every step and stores all the moves before adding them. The loop only updates the current position, so it does less extra work. The exact timings will vary between runs.

Problem 2: Campus Bus Arrivals

```
set.seed(2201)
days <- 10000

morning <- rowSums(matrix(rpois(days * 3, 4), nrow = days))
peak_9am <- rnorm(days, mean = 45, sd = 4)
midday <- rowSums(matrix(rpois(days * 6, 15), nrow = days))
peak_4pm <- rnorm(days, mean = 55, sd = 5)
evening <- rowSums(matrix(rpois(days * 6, 9), nrow = days))

total <- morning + peak_9am + midday + peak_4pm + evening
mean(total)
```

```
[1] 255.8775
```

The estimated average is about **255.88 passengers per day**. As a check, the expected total is

$$3(4) + 45 + 6(15) + 55 + 6(9) = 256.$$

I used standard deviations 4 and 5 for the Normal draws, since the given variances are 16 and 25. The code simulates independent hourly counts for all days at once, without loops.

Problem 3: Linear Regression

(a)

```
url <- paste0(
  "https://raw.githubusercontent.com/rfordatascience/tidytuesday/",
  "master/data/2021/2021-03-02/youtube.csv"
)
youtube <- read.csv(if (file.exists("youtube.csv")) "youtube.csv" else url)

remove <- c("brand", "superbowl_ads_dot_com_url", "youtube_url",
  "id", "etag", "published_at", "title", "description",
  "thumbnail", "channel_title")
youtube <- youtube[, !names(youtube) %in% remove]
dim(youtube)
```

```
[1] 247 15
```

The cleaned data have **247 rows and 15 columns**. I removed the names, links, IDs, publication timestamp, and other fields that could identify an ad. I kept `year` for the regressions, along with `kind` and `category_id`, which are general categories rather than identifiers.

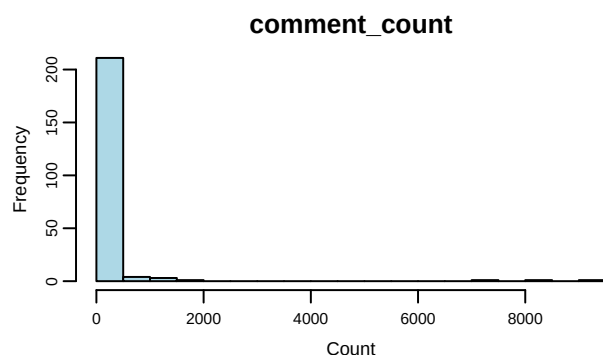
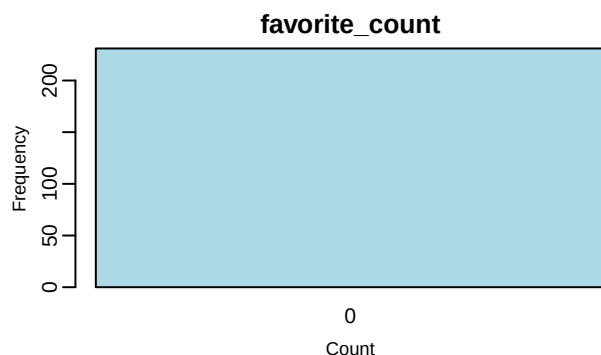
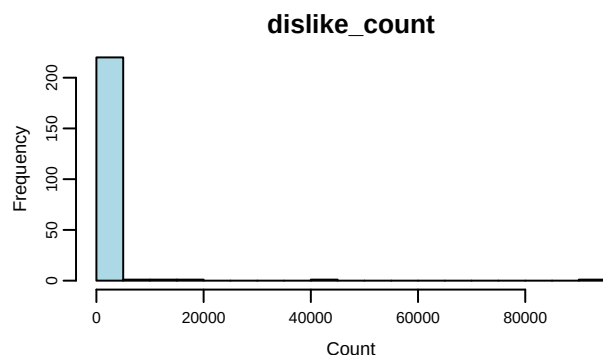
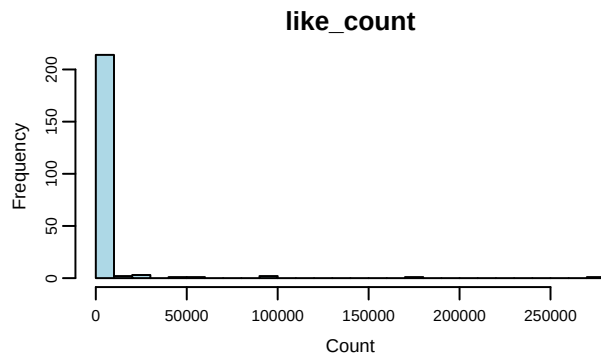
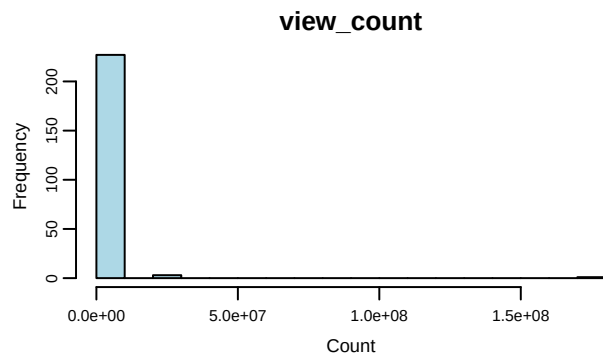
(b)

```
engagement <- c("view_count", "like_count", "dislike_count",
  "favorite_count", "comment_count")
summaries <- data.frame(
  outcome = engagement,
  observed = sapply(youtube[engagement], function(x) sum(!is.na(x))),
  zeros = sapply(youtube[engagement], function(x) sum(x == 0, na.rm = TRUE)),
  median = sapply(youtube[engagement], median, na.rm = TRUE),
  mean = sapply(youtube[engagement], mean, na.rm = TRUE),
  max = sapply(youtube[engagement], max, na.rm = TRUE)
)
knitr::kable(summaries, digits = 1, row.names = FALSE)
```

outcome	observed	zeros	median	mean	max
view_count	231	0	41379	1407556.5	176373378
like_count	225	9	130	4146.0	275362
dislike_count	225	47	7	833.5	92990
favorite_count	231	231	0	0.0	0
comment_count	222	41	10	188.6	9190

```
par(mfrow = c(3, 2), mar = c(3.5, 3.5, 2, 1),
  mgp = c(1.8, 0.55, 0), cex.lab = 0.85)
for (variable in engagement) {
  x <- na.omit(youtube[[variable]])
  if (variable == "favorite_count") {
    barplot(table(x), main = variable, xlab = "Count",
      ylab = "Frequency", col = "lightblue")
  } else {
    hist(x, breaks = 25, main = variable, xlab = "Count",
      col = "lightblue", cex.axis = 0.75)
  }
}
```

```
}
par(mfrow = c(1, 1))
```



Outcome	Choice	Reason
Views	Use <code>log(1 + count)</code>	A few ads have far more views than the rest.
Likes	Use <code>log(1 + count)</code>	Right-skewed, with some zeros.
Dislikes	Use <code>log(1 + count)</code>	Right-skewed, with many zeros.
Favorites	Exclude	All observed values are zero.
Comments	Use <code>log(1 + count)</code>	Right-skewed, with zeros and a few large counts.

I use `log1p()`, which calculates $\log(1 + \text{count})$. It reduces the skew and works when the count is zero. None of the outcomes is used as is.

```
outcomes <- setdiff(engagement, "favorite_count")
youtube[paste0("log_", outcomes)] <- lapply(youtube[outcomes], log1p)
```

(c)

I fit a separate model for each outcome-characteristic pair, with **year** included in every model. The binary predictors are coded as 0 and 1. Rows with missing model variables are left out by `lm()`.

```
features <- c("funny", "show_product_quickly", "patriotic",
             "celebrity", "danger", "animals", "use_sex")
youtube[features] <- lapply(youtube[features], as.integer)
results <- data.frame()
for (outcome in outcomes) {
  for (feature in features) {
    formula <- as.formula(paste0("log_", outcome, " ~ ", feature, " + year"))
    fit <- lm(formula, data = youtube)
    b <- summary(fit)$coefficients[feature, ]
    results <- rbind(results, data.frame(
      outcome = outcome, feature = feature,
      estimate = b[1], p_value = b[4], n = nobs(fit)
    ))
  }
}
knitr::kable(results, digits = 4, row.names = FALSE)
```

outcome	feature	estimate	p_value	n
view_count	funny	0.5298	0.2051	231
view_count	show_product_quickly	0.2090	0.6007	231
view_count	patriotic	0.1369	0.7839	231
view_count	celebrity	-0.0190	0.9629	231
view_count	danger	0.6668	0.0941	231
view_count	animals	-0.1733	0.6467	231
view_count	use_sex	-0.2963	0.4905	231
like_count	funny	0.3602	0.3401	225
like_count	show_product_quickly	0.1683	0.6405	225
like_count	patriotic	0.4931	0.2897	225
like_count	celebrity	0.2630	0.4779	225
like_count	danger	0.6253	0.0803	225
like_count	animals	0.0246	0.9428	225
like_count	use_sex	-0.3250	0.4001	225
dislike_count	funny	0.0415	0.8958	225
dislike_count	show_product_quickly	0.2429	0.4210	225
dislike_count	patriotic	0.6697	0.0856	225
dislike_count	celebrity	-0.2833	0.3616	225
dislike_count	danger	0.1851	0.5379	225
dislike_count	animals	-0.0738	0.7974	225
dislike_count	use_sex	-0.3889	0.2293	225
comment_count	funny	0.0475	0.8793	222
comment_count	show_product_quickly	0.3460	0.2469	222
comment_count	patriotic	0.5148	0.1685	222
comment_count	celebrity	0.2281	0.4556	222
comment_count	danger	0.1064	0.7210	222
comment_count	animals	-0.1918	0.4991	222
comment_count	use_sex	-0.3494	0.2735	222

None of the seven characteristics is significant at 0.05 for any of the four outcomes. There are therefore no significant positive or negative associations to report. For `log(1 + view_count)`, **danger** has the largest positive estimate: **0.6668**, with **p = 0.0941**. The estimated association is positive after adjusting for year, but it is not significant at 0.05.

(d)

danger also has the smallest p-value for view counts, so I use it here. The response is still $\log(1 + \text{view_count})$.

```
view_results <- results[results$outcome == "view_count", ]
selected <- view_results$feature[which.min(view_results$p_value)]
selected
```

```
[1] "danger"
```

```
columns <- c("log_view_count", selected, "year")
d <- youtube[complete.cases(youtube[, columns]), columns]
X <- cbind(1, d[[selected]], d$year)
colnames(X) <- c("(Intercept)", selected, "year")
y <- d$log_view_count
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
beta_hat
```

```
          [,1]
(Intercept) -31.39149243
danger       0.66678076
year        0.02065641
```

The three columns of X are the intercept, danger, and year. The calculation uses $\hat{\beta} = (X^T X)^{-1} X^T y$.

```
fit <- lm(log_view_count ~ danger + year, data = d)
cbind(matrix = as.vector(beta_hat), lm = coef(fit))
```

```
          matrix          lm
(Intercept) -31.39149243 -31.39149243
danger       0.66678076  0.66678076
year        0.02065641  0.02065641
```

```
all.equal(as.vector(beta_hat), unname(coef(fit)), tolerance = 1e-7)
```

```
[1] TRUE
```